

Chapter 1

Emissions Costs Calculation Documentation

1.1 Introduction

The `emissions.py` script calculates the societal costs of emissions due to line losses being compensated by generators. When transmission lines lose energy, generators must produce additional energy to compensate, resulting in additional emissions (CO₂, SO_x, NO_x) that have societal costs.

1.2 Method Overview

1.2.1 Purpose

This script calculates emissions costs by:

1. Calculating total energy losses (line losses + converter losses)
2. Determining total energy compensated (TEC) = losses × compensation percentage
3. Projecting energy mix evolution over project lifetime
4. Calculating emissions by pollutant for each year
5. Applying societal costs per kg of pollutant
6. Computing present value of lifetime emissions costs

1.2.2 Calculation Approach

The emissions cost calculation follows this structure:

$$\text{TEC} = \alpha \times \text{Total Energy Losses} \quad (1.1)$$

$$\text{Emissions}_k = \text{TEC} \times \sum_j (p_j \times I_{j,k}) \quad (1.2)$$

$$\text{Cost}_k = \text{Emissions}_k \times c_k \quad (1.3)$$

$$\text{Total Cost} = \sum_k \text{Cost}_k \quad (1.4)$$

$$\text{PV} = \sum_{t=1}^N \frac{\text{Annual Cost}_t}{(1 + r)^{t+\text{start_year}-1}} \quad (1.5)$$

where:

- α = compensation percentage (fraction of losses compensated)
- p_j = percentage of energy from source j in year t
- $I_{j,k}$ = emission intensity of pollutant k from source j (kg/MWh)
- c_k = societal cost per kg of pollutant k
- r = social discount rate

1.2.3 Inputs and Outputs

Inputs:

- Total energy losses (from `energy_losses.py`)
- Compensation percentage (fraction of losses compensated by generators)
- Energy source mix (initial percentages and growth/decay rates)
- Emission intensities by source and pollutant
- Societal costs per kg of pollutant
- Social discount rate
- Project lifetime, delay years, construction years

Outputs:

- Average annual emissions by pollutant (CO₂, SO_x, NO_x)
- Average annual costs by pollutant
- Lifetime total emissions and costs
- Present value of lifetime emissions costs

1.2.4 Integration with Other Scripts

This script integrates with:

- `energy_losses.py` — `get_total_energy_losses()` (loss data), `load_project_technical_details`
- `smart_loaders.py` — `load_emissions_details`, `load_financing_social_discount_rate`, `get_project_data_raw`
- `financial_utils.py` — `calculate_present_value`, `calculate_cod_year`
- `calculation_utils.py` — `from_percent`
- `smart_output.py` — `CTCCOutputManager` (`add_emissions_costs`, `write_batch_summary`)

1.3 Key Functions

1.3.1 Loss data from `energy_losses.get_total_energy_losses()`

The script does not compute losses itself; it calls `get_total_energy_losses()` from `energy_losses.py` and uses:

- `loss_data["total_losses_mwh_per_year"]` — total line + converter losses (MWh/year)
- `loss_data["project_lifetime"]` — project lifetime (years)

Project delay/construction years come from `load_project_technical_details()`.

1.3.2 `calculate_energy_mix_by_year(energy_source_mix_details, project_lifetime)`

Calculates energy mix for each year with growth/decay rates.

Parameters:

- `energy_source_mix_details` (dict): Dictionary with percentage and `rate_of_change` for each source
- `project_lifetime` (int): Number of years in project

Returns:

- `energy_mix_by_year` (list): List of dictionaries, one per year, with normalized percentages

Key Logic:

```
1 # sources: coal, oil, natural_gas, solar, wind, hydro, nuclear,
   # other
2 # Initial mix and rates from energy_source_mix_details (percentage,
   # rate_of_change)
3 initial_mix[source] = from_percent(details.get(source, {}).get("
   percentage", 0.0))
```

```

4 rates[source] = details.get(source, {}).get("rate_of_change", 0.0)
5 for year in range(1, project_lifetime + 1):
6     next_mix[source] = current_mix[source] * (1 + rates[source])
7     total = sum(next_mix.values())
8     normalized_mix = {s: next_mix[s] / total for s in sources} if
        total > 0 else next_mix
9     energy_mix_by_year.append(normalized_mix)
10    current_mix = normalized_mix

```

1.3.3 calculate_emissions_by_year(tec, energy_mix, emission_intensities)

Calculates emissions for a single year.

Parameters:

- **tec** (float): Total energy compensated in MWh
- **energy_mix** (dict): Dictionary of energy source percentages for the year
- **emission_intensities** (dict): Dictionary of emission intensities by pollutant and source

Returns:

- **emissions** (dict): Emissions in kg for each pollutant (CO₂, SO_x, NO_x)

Key Logic:

```

1 # Intensity keys: co2_intensity_kg_per_mwh, sox_intensity_kg_per_mwh
  , nox_intensity_kg_per_mwh
2 # Each maps source -> kg/MWh
3 intensity_key_map = {"co2": "co2_intensity_kg_per_mwh", "sox": "...",
  , "nox": "..."}
4 for pollutant in ["co2", "sox", "nox"]:
5     intensity_dict = emission_intensities.get(intensity_key_map[
        pollutant], {})
6     for source in sources:
7         p_j = energy_mix.get(source, 0.0)
8         I_jk = intensity_dict.get(source, 0.0)
9         emissions[pollutant] += total_energy_compensated_mwh * p_j *
            I_jk

```

1.3.4 calculate_lifetime_emissions(...)

Calculates emissions across project lifetime.

Parameters:

- **total_losses_mwh_per_year** (float): Total energy losses per year

- `compensation_percent` (float): Fraction of losses compensated
- `energy_source_mix_details` (dict): Initial mix and growth rates
- `emission_intensities` (dict): Emission intensities by source and pollutant
- `societal_costs` (dict): Costs per kg of pollutant
- `project_lifetime` (int): Project lifetime
- `social_discount_rate` (float): Social discount rate
- `delay_years` (float): Delay years
- `construction_years` (float): Construction years

Returns:

- Multiple values including yearly emissions, total emissions, average annual costs, lifetime costs, and present values

Key Logic:

```

1 start_year = calculate_cod_year(delay_years, construction_years)
2 total_energy_compensated_mwh = compensation_percent *
  total_losses_mwh_per_year
3 energy_mix_by_year = calculate_energy_mix_by_year(
  energy_source_mix_details, project_lifetime)
4
5 for year, energy_mix in enumerate(energy_mix_by_year, 1):
6     emissions = calculate_emissions_by_year(
7         total_energy_compensated_mwh, energy_mix,
8         emission_intensities)
9     year_cost = 0.0
10    for pollutant, emissions_kg in emissions.items():
11        cost_per_kg = societal_costs.get(f"{pollutant}_cost_per_kg",
12            0.0)
13        pollutant_cost = emissions_kg * cost_per_kg
14        year_cost += pollutant_cost
15        total_costs_by_pollutant[pollutant] += pollutant_cost
16    year_discount_year = start_year + year - 1
17    year_pv = year_cost / ((1 + social_discount_rate) **
18        year_discount_year)
19    lifetime_cost_pv += year_pv
20    # total_costs_by_pollutant_pv updated per pollutant similarly

```

1.3.5 main()

Main execution function that orchestrates the emissions cost calculation and output.

1.4 Example

The following example demonstrates a typical usage of the emissions cost calculation:

```
1 # Example: 500 MW Overhead AC line
2 #
3 # Inputs (from YAML):
4 #   - Total energy losses: 10,000 MWh/year
5 #   - Compensation percentage: 100% (all losses compensated)
6 #   - Initial energy mix:
7 #     * Coal: 20%, rate: -2%/year
8 #     * Natural gas: 30%, rate: -1%/year
9 #     * Solar: 15%, rate: +5%/year
10 #     * Wind: 20%, rate: +3%/year
11 #     * Other: 15%, rate: 0%
12 #   - Emission intensities:
13 #     * Coal: CO2=1000 kg/MWh, SOx=5 kg/MWh, NOx=3 kg/MWh
14 #     * Natural gas: CO2=400 kg/MWh, SOx=0.1 kg/MWh, NOx=0.5 kg/MWh
15 #     * Solar/Wind: CO2=0, SOx=0, NOx=0
16 #   - Societal costs:
17 #     * CO2: $0.05/kg
18 #     * SOx: $10/kg
19 #     * NOx: $15/kg
20 #   - Project lifetime: 50 years
21 #   - Social discount rate: 3%
22 #
23 # Calculation (Year 1):
24 #   TEC = 1.0 * 10,000 = 10,000 MWh/year
25 #   CO2 = 10,000 * (0.20*1000 + 0.30*400 + 0.15*0 + 0.20*0) = 3.2M
26 #       kg
27 #   SOx = 10,000 * (0.20*5 + 0.30*0.1) = 10,300 kg
28 #   NOx = 10,000 * (0.20*3 + 0.30*0.5) = 7,500 kg
29 #   Cost = (3.2M * $0.05) + (10,300 * $10) + (7,500 * $15) = $250
30 #       ,000/year
31 #
32 #   (Costs decrease over time as grid decarbonizes)
33 #
34 # Output:
35 #   - Average annual CO2: 2.5M kg/year
36 #   - Average annual cost: $150,000/year
37 #   - Lifetime cost: $7.5M
38 #   - Present value: $4.2M
```

Integration Example:

```
1 # When called from ctcc.py (main() builds results from
2 #   LifetimeEmissionsResults):
3 csv_manager = CTCCOutputManager()
4 results = {
```

```

4     "total_nominal": emissions_results.lifetime_cost,
5     "total_pv": emissions_results.lifetime_cost_pv,
6     "annual_cost": emissions_results.avg_annual_costs,
7     "co2_emissions_kg": emissions_results.total_emissions["co2"],
8     "co2_cost_nominal": emissions_results.total_costs_by_pollutant["co2"],
9     "co2_cost_pv": emissions_results.total_costs_by_pollutant_pv["co2"],
10    "sox_emissions_kg": ..., "sox_cost_nominal": ..., "sox_cost_pv":
11    ...,
12    "nox_emissions_kg": ..., "nox_cost_nominal": ..., "nox_cost_pv":
13    ...,
14    }
15 csv_manager.add_emissions_costs(results)
16 csv_manager.write_batch_summary()

```

1.5 Key Implementation Details

1.5.1 Compensation Percentage

The compensation percentage (α) represents the fraction of line losses that are compensated by generators. A value of 1.0 means all losses are compensated, while 0.0 means no compensation (no emissions cost).

1.5.2 Energy Mix Evolution

The script models how the energy mix evolves over the project lifetime using growth/decay rates. Each year, the mix is updated and normalized to sum to 100%. This captures grid decarbonization trends.

1.5.3 Emission Intensities

Emission intensities vary by energy source:

- **Fossil fuels** (coal, oil, natural gas): High CO₂, SO_x, NO_x
- **Renewables** (solar, wind, hydro): Zero emissions
- **Nuclear**: Zero CO₂, minimal SO_x/NO_x

1.5.4 Societal Costs

Societal costs represent the external costs of emissions (health impacts, climate change, etc.). These are typically higher than market prices and reflect the true societal impact.

1.5.5 Present Value Timing

Emissions costs start when the project becomes operational (after delay and construction periods). The present value calculation discounts annual costs back to the base year using the social discount rate.

1.5.6 Year-by-Year Calculation

The script calculates emissions and costs for each year individually, accounting for:

- Changing energy mix over time
- Time-varying discounting
- Pollutant-specific tracking